Improving Data Leakage Detection in Machine Learning Notebooks through Static Slicing and Structured LLM Prompts

TAHA DRAOUI, University of Michigan-Flint, USA
MOHAMED WIEM MKAOUER, University of Michigan-Flint, USA
CHRISTIAN NEWMAN, Rochester Institute of Technology, USA

Data leakage remains a critical yet under-diagnosed issue in machine learning pipelines, leading to inflated results and unreliable deployments. Existing detection approaches rely on static rules that often miss open-coded manipulations and fail to capture the diversity of real-world notebooks. This paper introduces a novel methodology that integrates static slicing with large language models (LLMs) to improve leakage detection. We use a Datalog-based static analysis that isolates compact, provenance-aware slices corresponding to model training and evaluation pairs, and we pair these with structured LLM prompts that guide step-by-step reasoning about potential leakage for each isolated slice. Evaluated on a curated benchmark of Python notebooks from Kaggle and GitHub, our approach achieves state-of-the-art performance in both preprocessing and overlap leakage detection, improving F1 scores over the previous state-of-the-art by 22% for preprocessing leakage and 15% for overlap leakage. Beyond these improvements, our slicing-based methodology substantially outperforms end-to-end prompting, demonstrating that precise program slicing is key to enabling LLMs to reliably detect leakage. Our findings highlight the effectiveness of combining program slicing and prompt engineering for data leakage detection and establish the first systematic LLM-based solution for detecting data leakage in machine learning code.

CCS Concepts: • **Software and its engineering → Software maintenance tools**; *Software verification and validation*; *Software testing and debugging*.

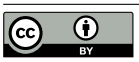
Additional Key Words and Phrases: Data Leakage, Machine Learning, Jupyter Notebooks, Large Language Models

ACM Reference Format:
Taha Draoui, Mohamed Wiem Mkaouer, and Christian Newman. 2026. Improving Data Leakage Detection in Machine Learning Notebooks through Static Slicing and Structured LLM Prompts. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE192 (July 2026), 22 pages. https://doi.org/10.1145/3808199

1 Introduction

Data leakage is a long-standing challenge in applied machine learning, with consequences that range from inflated benchmark results to flawed scientific claims and unreliable deployed systems [17, 19, 41]. Leakage arises when information from the test set improperly influences training, leading to overly optimistic performance estimates. Two prevalent forms are well documented in the literature. *Preprocessing leakage* occurs when statistics or transformations such as scaling, imputation, or feature selection are fit on the full dataset and later reused during evaluation. *Overlap leakage* arises when train and test sets share duplicated or near-duplicated samples, often because resampling is

Authors' Contact Information: Taha Draoui, University of Michigan-Flint, Flint, USA, tahadr@umich.edu; Mohamed Wiem Mkaouer, University of Michigan-Flint, Flint, USA, mmkaouer@umich.edu; Christian Newman, Rochester Institute of Technology, Rochester, USA, cdnvse@rit.edu.



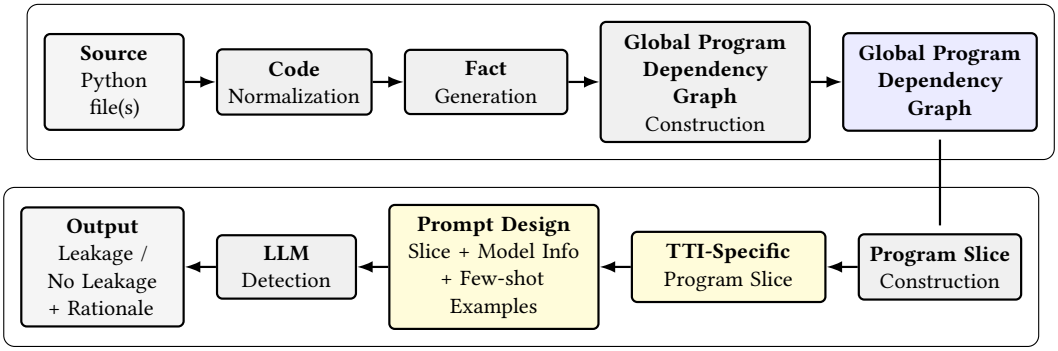
ACM 2994-970X/2026/7-ARTFSE192
https://doi.org/10.1145/3808199

applied before splitting or because of faulty split logic. The effects of undetected data leakage are far-reaching. Kapoor et al. document leakage affecting 294 published papers across 17 scientific fields, where flawed scientific claims were reused and propagated, while Yang et al. [41] report that nearly one-third of over 100,000 GitHub notebooks exhibit leakage, including in widely used repositories. Similar issues have been observed in machine-learning competitions, data science platforms, and production systems, where entire pipelines had to be revised after leakage was discovered [19, 41]. These findings indicate that leakage is not an isolated artifact of poor practice, but a recurring problem spanning both scientific research and everyday data science workflows. Because of their widespread adoption [24, 30] and their role as a primary medium for developing and sharing machine-learning pipelines, Jupyter Notebooks are a particularly important setting for early leakage detection. Notebooks frequently evolve into scripts, services, production pipelines, or tutorials; detecting leakage at this stage therefore has downstream benefits for reliability, reproducibility, and quality assurance across broader software engineering contexts and communities.

Nevertheless, leakage remains difficult to detect in practice. The issue is often referred to as a "silent bug" [11], a design flaw that does not trigger errors and may not manifest until deployment. Detecting it during development requires reasoning about code structure and dataflow. Manual inspection of notebooks is tedious and error-prone, particularly in large codebases. Automated solutions have therefore become a focus of research [6, 8, 11, 35]. Runtime approaches, such as those of Chorev et al. [8], identify anomalies and overlaps from observed data and model behavior but are limited to the datasets available at execution time. Static analysis methods, including NBLyzer [35] and its abstract interpretation extension [11], instead reason symbolically about code structure and tainted data sources. Yang et al. [41] improved on this direction by introducing the notion of *Train-Test Instances* (TTIs), pairs of training and evaluation calls that define localized contexts for detecting leakage. However, these methods share a common limitation: they rely on hand-crafted, API-specific rules. While effective when leakage arises from libraries and idioms known to their rules, they fail to capture the diversity of real-world notebooks, where leakage may be embedded in open-coded data manipulation or advanced routines.

The rise of Large Language Models (LLMs) suggests a promising alternative. LLMs have demonstrated strong capabilities in undertaking numerous tasks traditionally reliant on human input [2, 7, 18, 20, 21, 23, 25, 26, 28, 29, 34, 36, 37]. These LLMs are becoming supportive tools for developers in a variety of software engineering activities, including code quality improvement [15, 39], code repair [13, 33], and program comprehension [5, 22], and code generation [10, 12]. This flexibility makes them appealing for leakage detection, precisely where static rules often break down. Yet applying LLMs directly to full notebooks introduces new challenges. Notebooks typically combine exploratory analysis, visualization, and multiple overlapping pipelines, which obscure the exact flows of training and testing data. Unguided LLMs are easily distracted by this noise, resulting in both missed detections and spurious flags.

In this work, we combine the structuring power of static analysis with the learning ability of LLMs. Rather than tasking the LLM with parsing entire notebooks, we first use static analysis to isolate the exact code regions that define each TTI, yielding compact, provenance-aware slices. We then design prompts that guide the LLM to reason systematically over these slices, anchoring the analysis on explicit training and evaluation calls, incorporating step-by-step instructions, and, when beneficial, augmenting with illustrative few-shot examples. This straightforward and effective division of labor—static slicing for structure, LLM prompting for judgment—shrinks the search space, reduces noise, and ensures consistent reasoning. As we show in later sections, this approach achieves state-of-the-art performance in both preprocessing and overlap leakage detection, improving F1 scores over the previous state-of-the-art by 22% for preprocessing leakage and 15% for overlap leakage. Beyond these improvements, our slicing-based methodology substantially outperforms



Legend: Gray = analysis stages; Blue = graphs; Yellow = slices/prompts.

Fig. 1. Approach Overview

end-to-end prompting, demonstrating that precise program slicing is key to enabling LLMs to reliably detect leakage. This paper makes the following contributions:

- **A structuring-first approach to leakage detection.** We propose a pipeline that combines static slicing with LLM prompting. By isolating code into compact, provenance-aware *Train-Test Instance* slices, we enable LLMs to reason in focused contexts rather than across noisy notebooks. We have made our tool and curated dataset publicly available [1].
- **Prompt designs tailored for leakage reasoning.** We develop structured prompts that anchor analysis on explicit training and evaluation calls, guide step-by-step reasoning about data transformations, and incorporate few-shot examples when beneficial.
- **A new benchmark for leakage detection.** We construct and release an annotated dataset of real-world Kaggle and GitHub notebooks, labeled at the per-instance level for both preprocessing and overlap leakage.

2 Methodology

Notebooks often combine multiple models, ad-hoc preprocessing routines, and exploratory code cells, which makes it challenging to reliably identify train–test pairs and reason about potential leakage. Passing the full notebook to an LLM overwhelms the model and dilutes its ability to analyze specific dataflows. To study this problem, we first collect and annotate a set of real-world notebooks, which provides the basis for extracting *Train–Test Instances* (TTIs) and evaluating our approach. For a given notebook, our methodology then proceeds by normalizing the code and constructing a program dependency graph through static analysis. From this graph, we extract compact, TTI-specific program slices that capture only the instructions and value flows relevant to each train–test pair. These slices are combined with structured prompts—augmented with model information and few-shot examples and then passed to an LLM for leakage detection. The LLM produces a verdict together with a rationale, enabling fine-grained identification of leakage cases. Figure 1 summarizes these main stages.

2.1 Data Collection and Annotation

We first collect and annotate a dataset of real-world machine learning notebooks, serving as the foundation for assessing our leakage detection model. We start by reusing the dataset from Yang et

[1] Replication package

al. [41], which contains 100 reference GitHub notebooks from Yang et al. (specifically, 54 actual notebooks with at least one model pair). To assess the performance of our model and to compare with existing studies using a larger set, we augmented the existing set by sampling an additional 151 notebooks from GitHub. We also sampled 246 notebooks from Kaggle, given its popularity among data scientists [27], bringing the total to 451 notebooks. Across all collection phases and data sources, we filter out exploratory notebooks that don't train any ML models.

We calculated the margin of error (MoE) separately for each data source using a 95% confidence interval and assuming maximum variability ($p = 0.5$), following the standard formula for finite populations: $MoE = Z \times \sqrt{\frac{p \cdot (1-p)}{n}} \times \sqrt{\frac{N-n}{N-1}}$. Here, N is the population size, n the sample size, and Z the Z-score (1.96 for a 95% confidence interval). To combine margins of error from both Kaggle and GitHub samples into a single representative margin of error, we used the Root Sum of Squares (RSS) method: $MoE_{\text{combined}} = \sqrt{(MoE_{\text{Kaggle}})^2 + (MoE_{\text{GitHub}})^2}$. Our resulting testing set of 458 notebooks achieved a combined margin of error of **8.79%** at a 95% confidence level.

For the sampled notebooks, we performed annotations at a more granular level—specifically, the *Train-Test Instance (TTI) level*. From the sampled notebooks, we annotated 568 TTIs across 451 notebooks (each notebook can contain one or more TTIs). Annotations were conducted in 6 distinct rounds, each involving the labeling of approximately 100 TTIs. Each TTI was independently annotated by two primary authors, and disagreements were resolved through detailed discussion to reach a consensus. We measured inter-annotator agreement using Cohen's Kappa, consistently obtaining a high Kappa value of **0.933** or higher. This indicates robust consistency and reliability across annotations. In this work, we focus on detecting data leakage only when it propagates into a model's training or evaluation step.

2.2 Code Normalization and Intermediate Representation

The pipeline begins by transforming Python source code into a normalized Intermediate Representation (IR) inspired by Static Single Assignment (SSA) form [9]. This is important because the same logical operation can be expressed in many syntactic forms in Python, which makes direct analysis unreliable. By normalizing the code, we eliminate such variability and enable consistent downstream reasoning. The transformation is implemented as a single-pass AST-to-AST rewrite that overrides more than 40 Python AST node types and systematically rewrites executable constructs. We reuse the existing leakage detector infrastructure by Yang et al [41] to perform this transformation.

The resulting IR satisfies six invariants: (1) cach assignment introduces a fresh SSA version of the target variable; (2) compound assignments (e.g., tuple unpacking, starred targets, multi-target assignments) are decomposed into sequences of single-target assignments; (3) all non-atomic expressions are lifted into explicitly named temporaries, yielding a three-address-style representation; (4) control-flow merge points (e.g., after conditionals and loops) introduce explicit ϕ-functions to reconcile variable versions; (5) heap updates (attribute writes, subscript assignments, and selected in-place method calls) are rewritten into explicit functional-style wrapper calls that return updated base objects; (6) each IR statement retains a mapping to its original source line for traceability.

While classical SSA primarily enforces single assignment and ϕ-insertion at control-flow joins, this IR extends SSA with Python-specific normalization rules that eliminate implicit data flow arising from in-place updates and method side effects (e.g., a call such as `model.fit(X, y)` is rewritten into an explicit assignment `model_0 = model.fit(X, y)`, materializing the updated model state as a fresh SSA version). The result is a semantically equivalent but structurally linearized

program in which every data dependency is made explicit and uniformly representable as relational facts which eases the construction of data dependency relationships.

Figure 2 illustrates these invariants concretely. For example, the tuple assignment `Xtr, Xte, ytr, yte = train_test_split(...)` is decomposed into a temporary capturing the call result, followed by four explicit single-target assignments, each introducing a fresh variable version. Similarly, the method call `rf.fit(Xtr, ytr)` is rewritten so that its result is materialized in a fresh variable (e.g., `rf_0`), making the data flow from the training operation explicit. Intermediate constants and sub-expressions are lifted into temporaries (e.g., `_var16`, `_var17`), ensuring that every operand in the IR is either a name or a constant. The resulting IR serves as the direct input to the subsequent fact extraction stage.

```
1  Src 10: X_scaled = scaler.fit_transform(X)
2  Src 11: Xtr, Xte, ytr, yte =
   train_test_split(X_scaled, y, test_size
   =0.2, random_state=42)
3  Src 12: rf = RandomForestClassifier(
   random_state=42)
4  Src 13: rf.fit(Xtr, ytr)
5  Src 14: predictions = rf.predict(Xte)
6  Src 15: unrelated = predictions.sum()
       # unrelated to TTI anchors
7  Src 16: print("unrelated:", unrelated)
      # should be excluded from TTI slice
```

(a) Original source code

```
1  IR 18: X_scaled = scaler.fit_transform(X)
2  IR 21: (_v0, _v1, _v2, _v3) =
   train_test_split(X_scaled, y, test_size=
   _var16, random_state=_var17)
3  IR 22: Xtr = _v0
4  IR 23: Xte = _v1
5  IR 24: ytr = _v2
6  IR 26: _var18 = 42
7  IR 27: rf = RandomForestClassifier(
   random_state=_var18)
8  IR 28: rf_0 = rf.fit(Xtr, ytr)
9  IR 29: predictions = rf_0.predict(Xte)
10 IR 30: unrelated = predictions.sum()
11 IR 31: _s0 = "unrelated:"
12 IR 32: print(_s0, unrelated)
```

(b) SSA form

Fig. 2. Running example: SSA transformation

2.3 Fact Generation

Our implementation follows a Datalog-based static analysis architecture. Datalog has been widely used in program analysis due to its support for recursive data-flow reasoning and interprocedural modeling [1, 3, 32, 38]. In Datalog, program properties are represented as predicates (also called facts) and program dependency is derived via declarative inference rules. Formally, a predicate P of arity k is a relation over a domain $\mathcal{D}$:

$$P(t_1, \ldots, t_k) \quad \text{where } (t_1, \ldots, t_k) \in \mathcal{D}^k$$

Each fact corresponds to a ground instance of such a predicate and encodes a specific program property. New relations are derived through inference rules of the form:

$$H(\vec{x}) \leftarrow B_1(\vec{y}_1),\ B_2(\vec{y}_2), \ldots, B_n(\vec{y}_n)$$

where the head predicate H holds whenever all body predicates $B_1, \ldots, B_n$ are satisfied. To generate the initial set of facts representing the notebook's source code, the fact generator performs a single traversal over the SSA-based IR and emits typed relational tuples for each executable construct encountered. In total, the schema comprises 40 distinct facts, which can be grouped into six semantic categories:

Data-flow relations, capturing variable definitions and value propagation (e.g., `AssignVar`, `AssignBinOp`, `ActualReturn`). Intuitively, these relations record how values are created, transformed, and passed through the program, allowing us to trace where a piece of data comes from and how it changes over time.

Invocation and call-graph relations, modeling call sites, argument passing, and execution order (e.g., `Invoke`, `ActualParam`, `NextInvoke`, `InvokeLineno`). In simple terms, these relations describe how the program moves between functions and methods, which calls lead to which others, and in what order computations may occur.

Heap and data-structure relations, representing field accesses, indexing, slicing, and allocation events (e.g., `LoadField`, `StoreIndexSSA`, `Alloc`). These relations capture how data stored inside objects, arrays, and containers is read and modified, which is essential for understanding in-place updates common in ML pipelines.

Control-flow structure relations, encoding loop and branch boundaries (e.g., `For`, `While`, `If`, `OrElse`). Intuitively, these relations describe the possible execution paths of the program, such as whether a statement runs conditionally, repeatedly, or as part of an alternative branch.

Scope and structural relations, associating variables with enclosing methods or classes (e.g., `VarInMethod`, `LocalMethod`, `LocalClass`). These relations indicate where variables and statements logically belong in the program, helping distinguish local behavior from code defined in other functions or classes.

Type and hierarchy relations, derived from static type maps and class inheritance (e.g., `VarType`, `SubType`). In practice, these relations provide coarse-grained information about what kinds of objects values represent and how different classes relate to one another, supporting more informed reasoning about method calls and data usage.

By representing program behavior as relational tuples rather than syntax trees, we decouple semantic reasoning from Python-specific syntax and obtain a declarative substrate suitable for recursive inference and program dependency graph construction. This fact generation process is inherited from the previous leakage detector's infrastructure from Yang et al[41], to which we make 2 additions. First, we introduced consistent line-level annotations across nearly all fact types (e.g., in the form of a predicate argument), covering assignments, indexing operations, wrapper calls, and method definitions. These annotations allow later stages to easily recover the source lines associated with each node in the dependency graph. Second, the control-flow-oriented predicates (e.g., `For`, `While`, `If`, `OrElse`) are added by our implementation to support program slicing. The original detector's implementation supports data flow-only analysis and that is why we extended the generated base facts to capture control flow relationships.

2.4 Global Program Dependency Graph Construction

We use declarative Datalog rules to construct a program dependency graph that captures both data and control relationships, which we later anchor at the TTI level. Our dependency analysis builds on the foundational static-analysis components of Yang et al.'s leakage detector pipeline [41]. However, that detector is designed to define leakage existence flags as predicates that fire whenever their corresponding leakage rules are satisfied. In contrast, our approach defines a single program-dependency predicate that triggers when the required data- and control-flow conditions are met, still expressed through recursive rules over the same base facts.

Data and Control Dependency analysis. Building on the base facts generated in Section 2.3, we derive a single, uniform predicate, `FLowVarTransformation`, that captures both data and control dependencies between program instructions. To illustrate the construction, consider a scenario of a

method call `f(a, b)` at a given program point. The fact-generation phase emits the relations

$$\texttt{Invoke}(i, f), \quad \texttt{ActualParam}(1, i, a), \quad \texttt{ActualParam}(2, i, b), \quad \texttt{ActualReturn}(0, i, v),$$

where v denotes the receiving variable. During Datalog inference, these base facts are combined to model that the value assigned to v depends on the invocation i and its arguments a and b. This dependency is expressed by one or more derived `FLowVarTransformation` facts (depending on the number of inputs and outputs), which abstract over the concrete calling structure while preserving dataflow semantics. We ground this process in our running example shown in Fig. 3. The invocation of `StandardScaler.fit_transform` at SSA instruction IR18 is represented via `Invoke`, `ActualParam`, and `ActualReturn` facts. From these, the inference derives flow-variable transformations `FLowVarTransformation(X_scaled, 18, X)` and `FLowVarTransformation(X_scaled, 18, scaler)`. These derived relations are later materialized as edges in the program dependency graph, as highlighted in the figure.

To enable this construction uniformly across the program's diverse transformations, we define a collection of Datalog rules that infer `FLowVarTransformation` facts from our rich set of base relations. We cover four broad classes of transformations. In particular, we capture (1) intraprocedural value transformations, such as assignments and expressions; (2) memory and container transformations, including field accesses, indexed updates, and slicing; (3) interprocedural transformations, connecting actual parameters, and return values across calls; and (4) control-dependence transformations, which encode the influence of branch and loop conditions on enclosed computations. By expressing all these cases, we obtain a uniform dependency representation that can be recursively closed and sliced at the telemetry-identified train/test instruction level.

This new analysis represents a deliberate shift from the base detector's design. First, we introduce 56 additional Datalog rules to derive the unified `FLowVarTransformation` relation. These rules build on the same base facts as the original analysis, augmented with newly introduced control-flow facts described in Section 2.3. The latter enable the modeling of control dependencies that were intentionally excluded from the base detector, which was designed as a data-flow-only leakage analysis rather than a general program slicer. In addition, we statically match six standard-library patterns with side effects (`list.append/extend`, `dict.update/add`, `fillna`) to ensure that their data dependencies are captured even in the absence of return values. While these rules introduce a small amount of manual specification, they target well-established and stable library behaviors that commonly occur in machine-learning pipelines.

This analysis also removes several rules from the reference detector. The original leakage detector's data-flow analysis [41] defines multiple semantically distinct flow predicates—such as `ParamToRetEquivFlow`, `ParamToParamFlow`, and `ParamToRetCondEquivFlow`—to model API-specific data-flow semantics (e.g., equivalence and derivation). These predicates encode manual knowledge about how data propagates between parameters and return values for a curated set of library calls and are primarily intended to support rule-based leakage flags via taint-tracking and leakage-detection rules (17 predicates in total). In contrast, our unified dependency analysis does not rely on such API-specific flow semantics. Because leakage detection is delegated to the downstream LLM, we omit these specialized predicates and instead construct program slices solely from the generic classes of data and control transformations captured in our pipeline. This design choice simplifies the analysis while preserving the dependencies necessary for faithful slicing.

Aside from these changes, we reuse all core rules from the detector's analysis [41] that support dependency tracking. This includes pointer analysis, call-graph construction, and context-sensitive modeling via invocation contexts (which we omit from our examples for simplicity). We also retain the model-mapping logic that identifies model instantiations together with their associated input

and output calls; this logic forms the basis for anchoring Train–Test Instances (TTIs) in our analysis (see Section 2.4).

TTI-data mapping. On top of the data- and control-dependence analysis, we reuse and extend the detector's Datalog-based data–model mapping to associate model instances with their corresponding training and evaluation data. This mapping is expressed declaratively through the inferred predicates `ModelPair` and `TelemetryModelPair`, which identify pairs of training and evaluation call sites that operate on the same model instance and are related through a feasible invocation path. Concretely, the analysis detects standard training and evaluation APIs (e.g., `fit`, `predict`, `score`, `evaluate`) and groups call sites by model identity using interprocedural data-flow reasoning. A training call is paired with an evaluation call if the evaluation is reachable from the training invocation along a valid invocation path in the interprocedural call graph, and no intervening training call on the same model instance is observed along that path. This notion of "following" is therefore semantic rather than textual: it is defined in terms of invocation reachability and execution ordering constraints encoded in the analysis, rather than source-code order. The resulting *Train–Test Instances* (TTIs) correspond to pairs of instruction nodes in the program dependency graph that are connected via both model identity and invocation-path relations. Each `TelemetryModelPair` thus serves as a semantically grounded anchor from which we perform backward reachability to isolate the slice relevant to a specific training– evaluation interaction, as illustrated in Fig. 3.

We maintain this strategy from the static detector of Yang et al. [41], recognizing its effectiveness in identifying leakage-relevant execution points across a wide range of machine-learning workflows (e.g., Scikit-learn, Keras, PyTorch). In addition, we extend the original mapping to capture cases in which evaluation occurs implicitly within a training call, such as `fit(..., validation_data=...)`, `fit_generator(.., validation_data=..)`, or GridSearchCV.fit().

While the baseline detector identifies the presence of validation data at the argument level, it does not construct TTIs for such cases due to the absence of an explicit evaluation invocation. We address this limitation by introducing synthetic evaluation anchors co-located with the training call, allowing these patterns to be represented uniformly as TTIs and incorporated into the same invocation-path–based slicing process.

2.5 Program Slice Construction

Existing general-purpose Python slicers and notebook-oriented slicing tools [14, 16, 31, 40] assume that a slicing criterion, such as a program location, variable, output, notebook cell, or a sink filter, is explicitly provided by the user, and then compute backward reachability with respect to that criterion. In our case, the criterion is the Train-test Instance, which is not provided beforehand, but rather computed by constructing interprocedural call graphs, tracking model identity, and reasoning about execution order as discussed in section 2.4. This makes the direct use of off-the-shelf slicers impractical. Beyond anchor identification, coverage considerations further motivate our design. Even if we calculate the Train–Test Instances and supplied them as an explicit criterion to an off-the-shelf slicer, we would still need to manually provide summaries and configurations of ML semantics, as slicers typically treat third-party library behavior conservatively by default (e.g., assume mutation of all args/receiver) and rely on user-supplied summaries, annotations, or configuration rules to precisely model domain specific semantics. In contrast, building on the infrastructure of Yang et al. [41] enables us to normalize necessary ML-style semantics directly as part of our analysis, including assignments, indexing and slicing operations, container mutations, interprocedural argument and return flows, control dependencies, and model state updates induced by training calls and in-place updates. As illustrated in Fig. 3, the global dependency graph, coupled with TTI locations, enables us to perform reachability from identified TTI anchors and extract

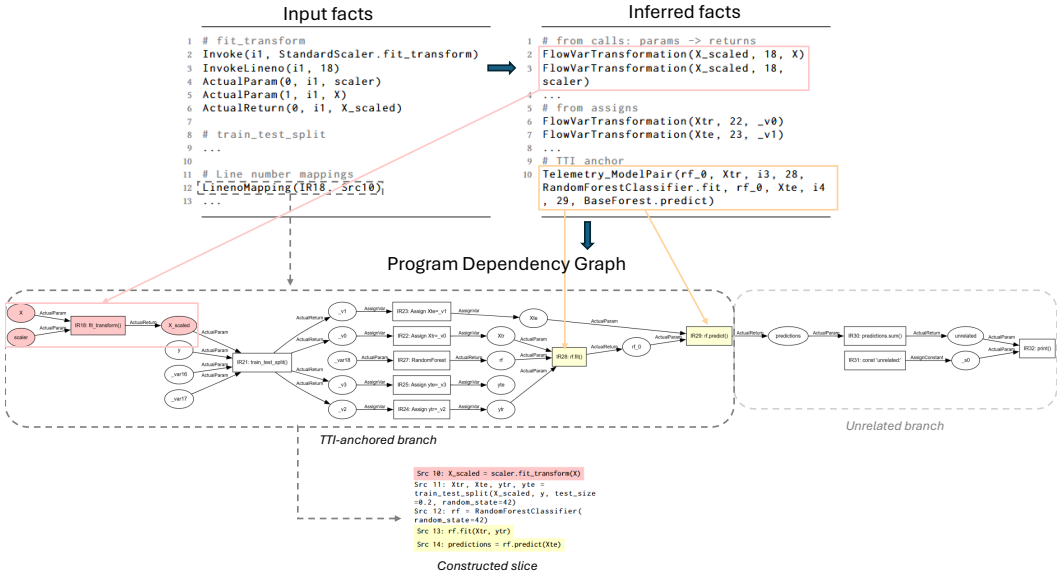


Fig. 3. Running example: Fact Generation, datalog analysis output, Program Dependency Graph, and constructure slice (in this example, we omit other information like call-site context sensitivity for simplicity)

only the subgraph relevant to each training–testing instance. Because we now localize detection to the level of each TTI and rely solely on graph connectivity. We translate each subgraph back into *source-level snippets*. This requires mapping from IR-level entities back to their original source lines, which we achieve using mapping facts recorded during IR generation (`LinenoMapping.facts`). We ensure that reconstructed snippets are syntactically complete and can be read in the same context as the original code. This dual representation supports both automated leakage detection and human inspection, closing the loop from static analysis to actionable findings. Moreover, this reorientation is particularly well-suited to LLMs. By discarding dead branches and unrelated operations, our pipeline exposes compact, faithful slices that capture the actual data dependencies and control structure for each TTI. These slices are precise enough to highlight leakage-prone flows, such as preprocessing unsplit data.

2.6 Prompt Design

2.6.1 Structured reasoning checklist. Once we extract the code slice for each model training and testing pair, we ask a Large Language Model (LLM) to analyze that slice and determine whether any data leakage is present. To guide this process, we design a structured prompt that includes explicit reasoning steps about leakage presence, which we call a structured reasoning checklist. The concrete checklist items are derived empirically during prompt tuning. The prompt-tuning set consists of approximately 130 TTIs drawn from the annotated dataset. To avoid bias, we post-filtered the prompt-tuning set so that notebooks used for tuning are exclusive to that set and do not appear in the evaluation data. We started with high-level prompts that only defined the leakage problem conceptually and did not give concrete steps to detect it. We observed that the LLM often relied on surface cues—such as the presence of a preprocessing operation—without verifying whether the transformed data actually flowed into the specific training and evaluation calls under analysis. To address this failure mode, we iteratively introduced checklist steps. These

steps basically instruct the LLM to start by locating where the train and test call is happening, then trace back data provenance. Concretely, the checklist consists of four reasoning steps, summarized in Table 1. Step 0 identifies the anchored training and evaluation calls, identifies input variables, and trace their provenance in the slice. Step 1 performs an early overlap check to detect cases where training and test data coincide or overlap at model training; this step explicitly anchors the LLM to validate obvious reuse before reasoning backward through the pipeline. If no overlap is found, Step 2 backtracks to identify candidate leaky transformations, and Step 3 determines whether any such transformations were fit on the full dataset prior to splitting. Finally, Step 4 confirms that the model actually consumes the contaminated variables identified in earlier steps.

Table 1. Structured reasoning checklist used in the LLM prompt for TTI-level leakage detection

Checklist Item	Purpose and Connection to Leakage Detection
Role and Inputs	Defines analysis context and role.
Step 0: Extract Train/Test Inputs	Provide training and evaluation call sites (line numbers and method names). Instruct LLM to identify its data variables and trace their provenance within the slice.
Step 1: Shortcut Overlap Check	Performs an early check for overlap leakage. If triggered, reasoning terminates.
Step 2: Identify Transformations	Detect transformations that may introduce leakage if fit on mixed data (e.g., imputation, scaling, encoding, feature selection, resampling).
Step 3: Verify Timing	Checks whether identified transformations were fit on the full dataset or prior to the train/test split, marking such cases as leakage candidates.
Step 4: Confirm Contamination	Verifies that the model's training or evaluation calls consume contaminated variables, grounding leakage in actual model usage.
Examples	Optionally augment with few shot examples.
Output Format	Requires a strict JSON output containing a leakage verdict and line-level explanations for automatic parsing and interpretability.

2.6.2 Few-Shot Example Selection. To enhance performance, we optionally augment the prompt with a small number of in-context examples illustrating leakage scenarios. These examples are retrieved from a fixed example bank constructed independently of the evaluation set. The example bank consists of short, self-contained code snippets paired with leakage verdicts and explanations (around 330 examples in total). The snippets are synthetically generated using an LLM (GPT-4o) to illustrate representative misuse patterns of common preprocessing and data-splitting operations, and the leakage labels and explanations are manually verified. Both the generation of these examples and the retrieval triggers used to select them rely on a shared vocabulary of transformation APIs. This vocabulary was based on the API catalog of the prior leakage detector [41]. In other words, the same third-party API catalog serves as (i) the basis for constructing illustrative leakage examples and (ii) the trigger set used for matching examples to a given snippet. Importantly, these API triggers serve only to retrieve illustrative examples and do not encode leakage rules or determine the final verdict, which remains entirely dependent on the LLM's reasoning over the statically sliced code.

3 Experimental Results

The goal of this experimentation is threefold: (i) to evaluate whether slicing improves coverage of train–test instances and instance-level accuracy, (ii) to identify the most effective prompt configuration, and (iii) to test whether our best setup outperforms a strong static baseline. To this

Table 2. Slicer Pipeline Runtime Breakdown

Runtime (s)	IR-gen	Type-Inf	Fact-gen	Datalog	Slice-ext	Total
Mean (std)	0.02 (0.03)	9.02 (3.06)	0.03 (0.03)	0.47 (0.38)	4.76 (12.36)	14.30 (12.73)

Table 3. GPT-3o-mini Latency, Token Consumption, and Cost per Request

	Latency (s)	Completion	Prompt	Cost
Mean (std)	5.68 (3.75)	569 (469)	2309 (462)	$0.005 ($0.002)

end, we partitioned our annotated dataset into three subsets: a prompt-tuning set of 130 examples (for prompt design, see Section 2.6), an evaluation set of 200 examples (for comparing prompt variants and slicing coverage against an end-to-end system), and a 200-example holdout set (for a head-to-head comparison with static rules). Both evaluation and holdout sets reflect realistic conditions, with preprocessing leakage in about 30% of cases and overlap leakage in about 20%. This setup enables us to evaluate the role of slicing, the impact of prompt design, and the benefits of our approach compared to static analysis. All experiments use the same base model, `GPT-o3-mini`, which is OpenAI's lightweight model selected for its strong performance on code understanding and reasoning tasks. This choice allows us to evaluate the impact of slicing and prompt design independently of model scale or specialized capabilities. We measure the full end-to-end pipeline. Table 2 reports a runtime breakdown across the validation notebooks: the mean end-to-end processing time is 14.3 seconds per notebook, with most variability arising from slice extraction. This reflects the long-tailed distribution of model-pair counts in real notebooks, where rare outliers with 50+ pairs inflate the upper tail. For the LLM stage, Table 3 reports average latency and token usage per TTI. Each invocation incurs an average latency of 5.68 seconds and a cost of approximately $0.005, which we find to be practically acceptable for interactive analysis.

3.1 RQ1: Does Static Slicing Improve Train-Test Instance Detection and Leakage Classification?

Approach. This experiment evaluates whether isolating each model's training and testing pipeline using static slicing improves the LLM's ability to detect leakage. Specifically, we ask whether breaking a notebook into focused code snippets for each Train–Test Instance (TTI) helps both to find more TTIs and to more accurately classify them as leaky or not. When the LLM receives an entire notebook (we will refer to it as an end-to-end LLM), it must locate all TTIs and judge whether their preprocessing leaks information, but the presence of multiple pipelines, exploratory code, or reused variables can distract or confuse the model. By contrast, in the slicer+LLM setting, we first extract each TTI into a small, focused snippet containing only the relevant training and testing logic, and then prompt the LLM once per snippet to classify it. Both approaches use the same base model and decision rules for leakage, differing only in input structure: full notebook versus isolated snippets. We selected ten notebooks from our validation set, containing a total of 55 ground-truth TTIs annotated with training and testing lines. For evaluation, we count the number of TTIs the two systems correctly identified if the predicted training and testing locations overlap the ground truth, and for those TTIs, we compute precision, recall, and F1 score in both preprocessing and overlap leakage settings. We then report the average performance of the leakage detection and TTI coverage.

Results. As shown in Table 4, the end-to-end LLM successfully retrieved 30 of the 55 TTIs, yielding a recall of 54.5%. We take a deeper look at the identified TTI and find that, although limited in coverage, the end-to-end LLM achieved a high precision of 91%, with 30 out of the 33 predicted

TTIs corresponding to actual train–test Instances present in the code. Among the three incorrect predictions, two were due to minor line number misalignments—where the predicted test line was one line above or below the actual call—while the third was a rare false positive where the predicted line did not correspond to any legitimate model usage. These results, however, only tell part of the story. While this high precision demonstrates that the LLM is capable of learning the pattern of model usage, its limited recall requires deeper investigation. Our qualitative analysis suggests that the LLM's coverage is influenced not by a single dominant factor, but by the interaction of three overlapping variables that revolve around complexity: (1) the distance between training and testing calls within a train–test instance, (2) the number of distinct models present in a notebook, and (3) the number of train–test instances associated with each model.

Table 4. Comparison of TTI coverage and leakage detection accuracy. Results are averaged across preprocessing and overlap leakage detection, considering both coverage and classification. Precision, recall, and F1 are computed only over correctly identified pairs

Method	TTI Coverage	Precision	Recall	F1 Score
End-to-end LLM	30/55 (54.5%)	0.50	0.75	0.60
Slicer + LLM (ours)	**55/55 (100%)**	**0.96**	**1.00**	**0.98**

We begin with the first factor: the distance between the train and test calls. In cases where this distance is minimal—such as when the test call directly follows the train call—the LLM is reliably able to detect the TTI. For instance, in notebooks like `2021-09-14-nb_65`[2] and `nb_196`[3], where train and test calls are only one invocation apart, all train–test instances are successfully retrieved. This confirms that spatial locality helps reduce ambiguity in TTI detection. Next, we consider the second factor: the number of distinct models in the notebook. As the number of models increases, even when each has only one train–test instance, the LLM's recall starts to drop. For example, in `nb_197`[4], which contains at least eight different models, only four train–test instances are detected—even though many of them are short in distance. This suggests that the presence of multiple models creates a form of contextual interference that dilutes the LLM's ability to match specific train and test calls accurately. Third, we examine the effect of multiple train–test instances associated with a single model. In notebooks like `andy6804tw_digit-recognizer`[5], the same model is used repeatedly for evaluation at different points, leading to several train–test instances. Despite short distances between calls, the LLM misses some of these instances, indicating that intra-model repetition increases internal ambiguity and weakens TTI association. A more extreme case is `rvarvade_rain-prediction-australia`[6], where a single model has eight train–test instances, but only one is retrieved. Crucially, these three factors do not operate in isolation. In some notebooks, the LLM succeeds despite one adverse factor because the others remain simple. For example, in `srvkarn_animalpred`[7], all train–test instances are detected even though a single model is evaluated multiple times—because the distances are small and there are no other models to compete for attention. Conversely, in more complex notebooks that combine long invocation distances, multiple models, and repeated usage patterns, the LLM often fails to retrieve even nearby or simple instances.

[2] 2021-09-14-nb_65
[3] nb_196
[4] nb_197
[5] andy6804tw_digit-recognizer
[6] rvarvade_rain-prediction-australia
[7] srvkarn_animalpred

These qualitative observations are further reflected in the aggregate detection results shown in Fig. 4. The slicer-based approach tracks the ground-truth number of train–test instances perfectly thanks to its effectiveness and to the enhancements we introduced to it- We capture TTIs where the validation data is passed as a parameter or the split happened internally, cases that wouldn't have been captured without our extension (discussed in section 2.4)- while the end-to-end LLM exhibits an erratic pattern as notebook complexity increases. At low levels of complexity (one to three train–test instances), the end-to-end LLM often succeeds, consistent with our earlier examples where short invocation distances and few models facilitated detection. Beyond that point, however, the number of captured instances fluctuates unpredictably: in some cases, the model retrieves up to six instances, while in others with similar complexity it detects only one or two. This lack of a clear monotonic trend indicates that the number of train–test instances alone cannot explain recall, and that other structural factors—such as multiple models, repeated instances within the same model, and contextual nesting—interfere and compound as complexity increases. This layered interference effect reflects the end-to-end LLM's limited working memory and inability to maintain fine-grained associations across structurally complex code.

It is important to note, however, that the three factors we highlight—invocation distance, number of models, and number of train–test instances per model—are those most clearly observed in our analysis. They do not exclude the possibility of other structural properties further complicating detection. For instance, we encountered cases where train and test invocations were nested inside another method body, making the call site context one level deeper. Such nesting significantly increases difficulty: in nb_1182[8], four train–test instances occurred within another method, and the LLM detected only one of them, missing the other three. This illustrates that multiple dimensions of complexity—beyond the three core factors we discuss—can further dilute attention and reduce recall. Taken together, these findings suggest that recall failures are not random but are governed by interacting structural properties that exceed the LLM's contextual capacity. This motivates the use of static slicing techniques, which reduce contextual noise by isolating relevant code segments, thereby improving the LLM's focus and enabling more reliable retrieval of train–test instances.

We also evaluated the impact of slicing on the subsequent classification task, where the goal is to determine whether a slice exhibits data leakage. Here, the same LLM is used in two configurations: end-to-end (detecting train–test instances and classifying them in one step) and in our proposed pipeline (slicer-based detection followed by classification). As shown in Table 4, the slicer-based approach achieves perfect recall and a high F1 score, while the end-to-end LLM lags behind with an F1 of 0.60. These results confirm that improved coverage directly translates into more reliable classification with reduced noise in the produced slices, further reinforcing the benefit of isolating relevant code regions through slicing.

3.2 RQ2: Which Prompt Configuration Leads to the Best Leakage Detection Accuracy?

Approach. This research question examines how different prompt designs affect the accuracy of LLM-based detection of preprocessing and overlap leakage. Building on RQ1—which established the value of slicing notebooks into focused snippets—we now ask how best to guide the model when analyzing these slices. Each prompt includes the relevant code and asks the model to trace dataflows into training and testing, but we vary three design factors that give rise to several configurations. The *no checklist baseline* contains only the code, leaving all reasoning implicit. A *checklist configuration* augments the prompt with explicit reasoning steps but does not specify training or testing lines. To explore the role of anchoring, we consider two variants: one in which the LLM must identify training and testing calls itself (*self-guided start*) and another in which we

[8] nb_1182

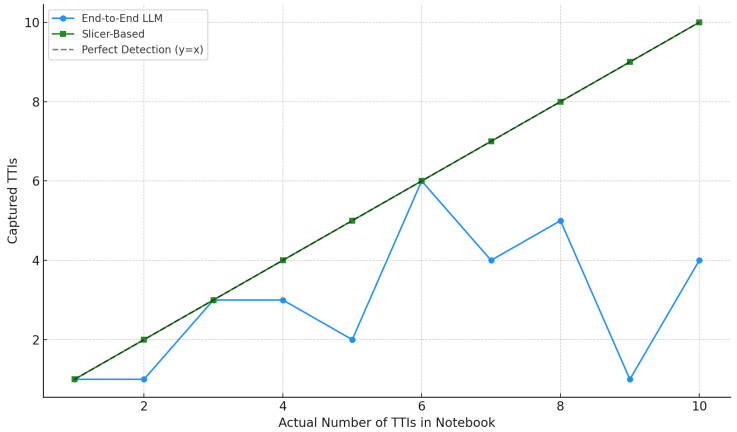


Fig. 4. TTI detection as notebook complexity increases. The slicer-based approach closely tracks the ground truth count, while the end-to-end LLM lags as the number of TTIs per notebook increases

Table 5. Performance of different prompt configurations for preprocessing and overlap leakage detection

Prompt Setup	**Preprocessing Leakage**			**Overlap Leakage**		
	Precision	**Recall**	**F1 Score**	**Precision**	**Recall**	**F1 Score**
No checklist	0.85	0.92	0.89	0.93	0.40	0.56
Checklist Only	0.94	0.92	0.93	0.93	0.74	0.83
Checklist + Self-Guided Start	0.92	0.95	0.94	0.90	0.80	0.85
Checklist + Line Hints	0.94	0.94	0.94	0.95	1.00	0.97
Checklist + Line Hints + Few shots (*LeakCheckLLM*)	**0.95**	**0.97**	**0.96**	**0.97**	**1.00**	**0.99**

provide the exact line numbers of these calls (*line hints*). Finally, a *few-shot configuration* extends the checklist-plus-hints setup with three labeled examples illustrating common leakage scenarios.

Our hypotheses are: (H1) structured reasoning checklists improve accuracy over unstructured prompts by reducing spurious flags and encouraging consistent analysis; (H2) explicitly anchoring the model on training and testing calls improves recall compared to leaving this identification implicit; and (H3) augmenting the prompt with few-shot examples further enhances performance compared to zero-shot prompts. These hypotheses will be evaluated by comparing the above prompt configurations on the same benchmark of annotated Train–Test Instances.

Results. Table 5 shows that prompt design has a dramatic effect on the model's ability to detect overlap leakage. With a prompt that only explains what leakage is and under what conditions it arises, but without prescribing reasoning steps, the LLM detects very few overlap cases (recall of 0.40), since it is not instructed to check systematically for situations such as the same data being reused for both training and testing. Once we introduce a structured checklist, recall jumps to 0.74, reflecting that the model now explicitly checks for this dominant pattern and can flag it reliably. Making the prompt even more explicit by anchoring the reasoning on the training and testing calls (the self-guided start) improves recall further to 0.80, because the LLM is now focused on tracing

how data flows into the model rather than scanning broadly. When we provide the model with the exact line numbers of these calls, recall reaches 1.00: the LLM no longer needs to infer where training and testing occur, and can devote its attention entirely to verifying whether the same data is reused and to tracing the associated pipeline. Finally, when this structured reasoning is combined with a few carefully chosen examples, the model achieves its best balance, not only maintaining perfect recall but also improving precision, since examples help it avoid spurious flags. Together, these results demonstrate that overlap leakage detection benefits most from explicit anchoring on the train–test instance, and that examples complement this by reinforcing the correct interpretation of subtle patterns.

Table 5 also shows that the LLM behaves differently when detecting preprocessing leakage. Even with an unstructured prompt, performance is already relatively strong (F1 of 0.89), because the model tends to over-flag suspicious transformations and thereby achieves high recall. Many of these false positives were, in fact, overlap leakage instances mislabeled as preprocessing leakage, indicating that without explicit guidance, the LLM conflates any suspicious reuse or transformation with preprocessing leakage. A structured checklist improves precision by reminding the model what counts as leakage and, just as importantly, what does not; this reduces spurious flags while preserving high recall (F1 rises to 0.93). Anchoring the reasoning to the training and testing calls further improves recall to 0.95 and F1 to 0.94, since the model can now trace preprocessing transformations back to the model's inputs more faithfully. Providing exact line hints here adjusted the precision recall balance by making the LLM be more precise when checking provenance and thus reducing false positives at the expense of being a little bit more conservative, which slightly dropped recall. Finally, adding few-shot examples produces the best results (F1 of 0.96), as the examples reinforce the checklist and hints with concrete illustrations of when preprocessing is or is not leaky. These results align with all three hypotheses: (H1) structured reasoning steps improve precision by curbing over-flagging; (H2) anchoring on training and testing calls improves recall by guiding the model to trace dataflows more systematically; and (H3) few-shot examples provide the strongest overall balance, raising both precision and recall. Our best-performing configuration, *LeakCheckLLM*, achieves near-perfect performance on both leakage types, with F1 of 0.96 for preprocessing and 0.99 for overlap, demonstrating that carefully designed prompts yield state-of-the-art detection across diverse leakage scenarios. We acknowledge that part of the effectiveness of this configuration is the precise few shot examples that do rely on some encoding of some library APIs but even zero-shot alternatives still offer very competitive performance.

3.3 RQ3: How Does Our Approach Compare to a Strong Static Baseline on a Held-Out Test Set?

Table 6. Per-TTI comparison by leakage type on the held-out test set

	Overlap			Preprocessing		
Method	P	R	F1	P	R	F1
Yang et al.[41]	0.88	0.73	0.80	0.76	0.64	0.70
LeakCheckLLM	**1.00**	**0.90**	**0.95**	**0.92**	**0.93**	**0.92**

Approach. Yang et al.'s [41] baseline is the only existing tool that performs leakage detection at the Train-Test Instance (TTI) level. It operates by encoding API-specific rules for common machine learning methods, making it a strong static reference point for comparison. We therefore evaluate LeakCheckLLM against this baseline on a disjoint test set of 200 notebooks, each annotated with

TTIs and labeled for both overlap and preprocessing leakage. Both systems are applied to the same notebooks to ensure a fair head-to-head comparison. Although our earlier results established the benefits of incorporating few-shot examples, in this experiment, we adopt a purely zero-shot setting in order to test the LLM's ability to recognize APIs and leakage-inducing manipulations natively without relying on handcrafted, pattern-based exemplars.

Results. The results in Table 6 show a clear advantage of LeakCheckLLM over the static baseline. For overlap leakage, LeakCheckLLM achieves perfect precision (1.00) and markedly higher recall (0.90 vs. 0.73), resulting in a substantially stronger F1 score (0.95 vs. 0.80). For preprocessing leakage, it also outperforms the baseline across all metrics, improving recall by nearly 30 percentage points (0.93 vs. 0.64) and raising F1 from 0.70 to 0.92. These gains demonstrate that our slicing-plus-LLM approach not only reduces false alarms but also captures a wider range of leakage cases, yielding consistent improvements across both leakage types.

Preprocessing Leakage. When analyzing false positives, we observe that our LLM-based approach and the static rules of Yang et al.[41] are misled by very different factors. Our method produced six false positives, and interestingly, most of them are not genuine preprocessing leakage but cases of *overlap leakage*. For example, in `2021-09-11-nb_86`[9] and `nb_1127`[10], the same data was reused for both training and testing, yet the LLM was distracted by surface-level patterns suggestive of preprocessing leakage. Specifically, the presence of leak-prone operations such as dummy column alignment or label encoding led the model to flag these examples incorrectly. If the model had prioritized the explicit shortcut provided in our prompt—to identify the data used for training and testing first and check if they are identical—it would have identified the true leakage as overlap rather than preprocessing. This illustrates that the model is occasionally over-attentive to cues that are correlated with leakage, rather than following our step-by-step verifications. In contrast, the static rules produced thirteen false positives, largely concentrated in cases where training and test sets were *imported independently*. In such situations, any preprocessing performed separately on each dataset—such as imputing missing values with `fillna`, label encoding, or group-based median filling—was mistakenly interpreted as if it had been applied to a single concatenated dataset prior to splitting. For example, in `akspan12_titanic-problem-final`[11] shown in listing 1, missing values in both the training and the test set are imputed separately using class-based medians:

Listing 1. Snippet from `akspan12_titanic-problem-final` where imputation is performed separately on training and test sets

```
train['Age'].fillna(train.groupby('Title')['Age'].transform('median'),
    inplace=True)
test['Age'].fillna(test.groupby('Title')['Age'].transform('median'),
    inplace=True)
train['Fare'].fillna(train.groupby('Pclass')['Fare'].transform('median'),
    inplace=True)
test['Fare'].fillna(test.groupby('Pclass')['Fare'].transform('median'),
    inplace=True)
```

This code is non-leaky, since training and test are never intermixed; however, the static rules flag it as leakage, highlighting all the lines of code where train columns are transformed with fillna. The same confusion arises in more compact forms, such as `for i in [train, test]` loops (where each dataset is imputed separately but grouped syntactically), which the static analysis mistakenly interprets as preprocessing on the concatenated dataset. These cases were effectively identified by the LLM. We note that this is not unique to preprocessing leakage: the same pattern also arises in

[9] 2021-09-11-nb_86
[10] nb_1127
[11] akspan12_titanic-problem-final

overlap leakage, and we will revisit this issue. Taken together, these findings demonstrate that while static rules are effective when the leakage pattern aligns closely with their encoded assumptions (e.g., a single dataset split into train and test), they fail to generalize to the broader spectrum of leakage scenarios found in real-world notebooks. Our LLM-based detector both avoids unnecessary false alarms and expands coverage to complex manipulations, making it a more practical and reliable choice for this domain. What unites both approaches is that false positives often arise when the detector fails to disambiguate between *legitimate, isolated preprocessing* and *contaminated preprocessing that leaks across the split*. For our LLM, the issue is over-reliance on surface-level leakage cues at the expense of cross-checking which dataset is ultimately used for training and testing. For the static rules, the issue is an inability to distinguish separate datasets from a single dataset processed prior to splitting. When analyzing false negatives, we find distinct failure modes for our approach and for static analysis. For our system, no single recurring pattern emerges among false negatives. With one exception, a straightforward case of preprocessing leakage that was surprisingly missed, the majority of false negatives are *long, complex, and context-heavy notebooks*. These cases often involve *function call chains* where preprocessing logic is hidden behind custom helper methods. As a result, leakage cues are embedded within otherwise safe transformations, making them less salient. This complexity, especially when preprocessing occurs at multiple call-sites, explains why the LLM struggles: the model fails to recognize leakage when identifiers are deeply nested in advanced or nuanced code structures rather than appearing in a simple linear pipeline. In contrast, static analysis fails systematically whenever an API is not explicitly modeled. Most of its false negatives fall into categories where leakage arises from *unmodeled preprocessing APIs*. For example, pipelines that apply `LabelEncoder` or `OneHotEncoder` across the full dataset leak category information, yet these encoders are not covered in the specification. Similarly, *feature selection leakage* escapes detection: correlation-based feature dropping and tools like `ELI5`'s feature selector are not represented in the rule set, even though they propagate test-set information into training. For instance, in the notebook `artgor_santander-eda-fe-fs-and-models`[12] shown in listing 2, static analysis did not capture the following case, whereas our system correctly flagged it:

Listing 2. Output of our LLM on an ELI5 feature selector leakage example missed by the static rules and flagged by our system in `artgor_santander-eda-fe-fs-and-models`

```
top_features = [i for i in eli5.formatters.as_dataframe.explain_weights_df(
    model).feature if 'BIAS' not in i][:100]  # <-- WARNING: The top feature selection is
    derived from a model that
# was trained using a split (which included both training and validation data),
# and its resulting feature importance is then used on the full dataset.

X1 = X[top_features]  # <-- WARNING: The selected top features  re extracted from the
entire
# dataset (X) before performing a new train-test split, thereby leaking
# test-set information into the training transformation.
```

This illustrates the difference between the two approaches: static analysis misses cases not covered by hand-written API rules, while our system flags them with contextual warnings. More broadly, our false negatives arise due to *structural and contextual complexity* (nested calls, nuanced pipelines), whereas static analysis false negatives stem from *coverage gaps* in the API specification.

Overlap Leakage. When comparing our overlap leakage detection against the static analysis rules of Yang et al. [41] we observe both areas of agreement and clear divergences in coverage. Both approaches share a small set of false negatives, such as the time-series cases `nb_229`[13] and

[12] artgor_santander-eda-fe-fs-and-models
[13] nb_229

2021-09-01-nb_2528[14], where sliding window construction introduces subtle overlap leakage. These cases are particularly challenging because the leakage does not arise from obvious preprocessing operations but instead from how temporal context is reused across training and testing, a pattern that requires reasoning about order and dependency rather than simple API misuse. Similarly, ambiguous cases like snehalshirgure_housing-price-prediciton-nb[15] produce confusion for both approaches, as the code contains both a leaky and a non-leaky path, making the ground truth itself less clear-cut. Where the approaches diverge is most revealing. Our LLM-based method successfully flags a wide range of overlap leakage cases that the static analysis systematically misses, especially when leakage is hidden within *advanced data manipulations* such as oversampling, augmentation, or concatenation across train and test sets. For instance, in the jonnyevans_multiclass-svm[16] notebook shown in listing 3, new synthetic samples are created by rotating images and writing them into preallocated placeholders in the dataset *before* performing the train–test split:

Listing 3. Snippet from jonnyevans_multiclass-svm showing augmentation before splitting

```
train = pd.read_csv('../input/train.csv')
X = train.values[:, 1:]
Y = train.ix[:, (0)]
n = len(Y)
random_indexes = np.random.randint(0, high=n, size=10000)

# Preallocate space for augmented samples
X = np.vstack((X, np. eros((10000, X.shape[1]))))
Y = np.concatenate([Y, np.zeros(10000)])

# Open-coded augmentation: rotate originals into new slots
for ind in random_indexes:
    img = X[ind, :].reshape(28, 28)
    angle = np.random.randint(- 0, 30)
    img_rot = rotate(img, angle)
    X[n, :] = img_rot.reshape(1, 784)
    Y[n] = Y[ind]
```

This procedure contaminates the test set with near-duplicates of training samples, since the augmented rows are mixed back into the same array before the subsequent split. Static rules remain blind to this pattern, not only because they lack value- and control-sensitivity, but more fundamentally because they rely on explicit syntactic markers (e.g., an augmentation API) to guarantee the presence of leakage. In contrast, the above code is entirely *open-coded*: its leaky nature emerges from the *sequence of operations*—preallocating space, looping over random indices, rotating images, and assigning them into new rows. Our LLM-based approach captures this intent holistically, recognizing that the transformation implements data augmentation before splitting. In contrast, static rules cannot "read between the lines" to infer this semantic intent. Similarly to preprocessing patterns, we also find that the static analysis occasionally *overflags* leakage in cases where training and test sets are imported independently. In such situations—illustrated by 2021-09-20-nb_1743[17]—test data is pre-processed separately but never intertwined with the training set. Notably, this is the same pattern we observed for preprocessing leakage, where independently imported training and test sets regularly mislead the static analysis. We therefore expect this behavior to be systematic across leakage types. While our approach correctly avoids

[14] 2021-09-01-nb_2528
[15] snehalshirgure_housing-price-prediciton-nb
[16] jonnyevans_multiclass-svm
[17] 2021-09-20-nb_1743

labeling this case as leakage, the static rules erroneously do so because they lack the capacity to distinguish isolated preprocessing from contaminated reuse in the described cases.

These results underscore the superiority of our technique while maintaining generality and scalability. This demonstrates that our method is not dependent on explicit pattern specifications and remains robust across prompting configurations, with only a handful of method calls explicitly modeled in our program dependency graph. Taken together, these results highlight the key advantage of our slicer+LLM pipeline: it not only captures cases that hand-crafted rules did not cover but also provides accurate, context-aware detection across diverse leakage scenarios. Unlike static analysis, which is constrained by coverage gaps in API specifications and prone to both over- and under-flagging, our approach adapts to open-coded and complex pipelines while avoiding spurious alarms. Even under zero-shot prompting, it consistently recovers leakage cases missed by the baseline and sustains state-of-the-art precision and recall. These findings establish our method as a more reliable and scalable solution for real-world notebooks, surpassing static rule-based detection both in breadth and in robustness.

4 Discussion

Our approach is effective because it grounds leakage detection in explicit data-flow dependencies before invoking an LLM, rather than relying on the enumeration of leakage patterns as static rules do. That said, LLMs still require guidance to perform leakage classification, which we provide through structured prompts. Our current prompts are designed specifically for preprocessing and overlap leakage, and we do not claim that they automatically generalize to all leakage types. Extending the approach to new leakage categories primarily requires defining additional prompt steps that explain the relevant concepts to the LLM. The LLM can then leverage its latent knowledge to reason about these scenarios. The slicer itself is agnostic to leakage type and can be applied to any leakage that manifests through code-level data-flow dependencies (e.g., misuse of feature selection or target leakage introduced by program operations). However, the approach does not generalize to leakage outside the program's data flow, such as dataset-level issues (e.g., duplicate samples in the input data), or intentional overfitting during model training.

We focus on notebooks in this work because they are the dominant medium for developing and sharing data science and machine-learning workflows in practice [24, 30] . This focus is also consistent with prior static leakage-detection works [8, 11, 35], which similarly target Jupyter Notebooks. Our evaluation demonstrates that, within this Notebook setting, the proposed analysis effectively identifies model training–evaluation pairs and extracts precise dependency slices for leakage detection. Our current slicer implementation operates on single-file Python artifacts. Within this scope, the approach is not inherently limited to notebooks and can be applied equally to Python scripts, provided they are generated from a single file containing all their dependencies. To support more environments, our tool would require additional program analysis infrastructure (e.g., inter-file call graphs and cross-module data-flow tracking), which we plan to implement in the future.

5 Related Work

Recently, several peer-reviewed studies have proposed methods for automatically detecting leakage. These techniques fall broadly into three categories: static code analysis, dynamic validation, and learning-based code classification. Yang et al. [41] introduce a static data-flow analysis that operates at the level of individual model instances. For each model in a notebook, the tool traces how training and test data are used, enabling detection of overlap leakage, preprocessing leakage, and multi-test reuse. They achieve this by modeling the API usage of common machine learning libraries. This approach was among the first to include a manually annotated benchmark dataset comprising 100

real-world notebooks. While their analysis operates at the per-model level, the annotated dataset they release is only labeled at the notebook level—indicating whether any leakage is present in the notebook as a whole. The analysis achieved a precision of 97.6% and a recall of 67.8%, reflecting notebook-level detection rather than per-model or per-leakage-type performance, which limits the granularity of evaluation. AlOmar et al. [4] present *LEAKAGEDETECTOR*, a PyCharm IDE plugin that integrates the static detection approach by Yang et al.[41]. Their contribution focuses on enhancing usability and developer support through quick fixes and IDE-based feedback for leakage instances.

5.1 Threats to Validity

There are several internal threats to validity. Our structured prompts and slicing strategies, while effective, may encode assumptions about what constitutes leakage, introducing prompt bias. In borderline cases such as ambiguous preprocessing steps, the model's decisions may reflect our design choices more than an objective notion of leakage. Similarly, the labeling of leakage at the Train–Test Instance level, though supported by inter-annotator agreement, remains inherently subjective: some transformations sit at the boundary of what can reasonably be considered leaky, and the resolution of annotator disagreements could affect the reported results. We also acknowledge threats to external and construct validity. Our benchmark notebooks, drawn from Kaggle and GitHub, may not fully represent leakage practices in industrial pipelines, limiting generalizability. Although our static analysis accounts for common libraries and idioms, custom or unseen preprocessing code could evade detection.

6 Conclusion

This paper presented a systematic approach to detecting and fixing data leakage in machine learning notebooks by combining static slicing with structured LLM prompting. Our slicer isolates Train–Test Instance slices, ensuring that the model reasons within focused, provenance-aware contexts rather than across noisy notebooks. We showed that reasoning steps, when made explicit in the prompt, substantially improve precision and recall by guiding the LLM to trace data dependencies and transformations consistently. Few-shot examples provide additional gains, though our experiments show that removing them does not drastically degrade performance, indicating robustness in the zero-shot setting. Finally, we demonstrated that LLMs, when equipped with these structural and prompting enhancements, can detect non-trivial leakage patterns, such as advanced feature-selection leakage or custom preprocessing that escapes static, rule-based analyzers. These findings highlight the importance of combining program analysis with prompt design and establish a foundation for future work applying this approach across files.

Acknowledgments

This research is funded by the National Science Foundation under Grant No. CNS-2525507.

Declaration of AI-assisted technologies

During the preparation of this work, the author(s) used *Grammarly* and *ChatGPT* to improve the paper's language and readability.

Data Availability

The replication package is available on GitHub[18].

[18] Replication package

References

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases: The Logical Level* (1st ed.). Addison-Wesley Longman Publishing Co., Inc., USA.

[2] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. 2023. Towards human-bot collaborative software architecting with chatgpt. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. 279–285.

[3] Nicholas Allen, Padmanabhan Krishnan, and Bernhard Scholz. 2015. Combining type-analysis with points-to analysis for analyzing Java library source-code. In *Proceedings of the 4th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis* (Portland, OR, USA) *(SOAP 2015)*. Association for Computing Machinery, New York, NY, USA, 13–18. doi:10.1145/2771284.2771287

[4] Eman Abdullah AlOmar, Catherine DeMario, Roger Shagawat, and Brandon Kreiser. 2025. LeakageDetector: An Open Source Data Leakage Analysis Tool in Machine Learning Pipelines. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 844–849. doi:10.1109/SANER64311.2025.00089

[5] Eman Abdullah AlOmar, Luo Xu, Sofia Martinez, Anthony Peruma, Mohamed Wiem Mkaouer, Christian D Newman, and Ali Ouni. 2025. ChatGPT for code refactoring: Analyzing topics, interaction, and effective prompts. In *2025 IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*. IEEE, 389–398.

[6] Nouf Alturayeif and Jameleddine Hassine. 2025. Data leakage detection in machine learning code: transfer learning, active learning, or low-shot prompting? *PeerJ Computer Science* 11 (2025), e2730. doi:10.7717/peerj-cs.2730

[7] Ali Ben Mrad, Abdoul Majid O. Thiombiano, Mohamed Wiem Mkaouer, and Brahim Hnich. 2024. Assessing Large Language Models Effectiveness in Outdated Method Renaming. In *International Conference on Service-Oriented Computing*. Springer, 253–260.

[8] Shir Chorev, Philip Tannor, Dan Ben Israel, Noam Bressler, Itay Gabbay, Nir Hutnik, Jonatan Liberman, Matan Perlmutter, Yurii Romanyshyn, and Lior Rokach. 2022. Deepchecks: A library for testing and validating machine learning models and data. *Journal of Machine Learning Research* 23, 285 (2022), 1–6.

[9] Ron Cytron, Jeanne Ferrante, Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. 1991. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 13, 4 (1991), 451–490.

[10] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration Code Generation via ChatGPT. *arXiv preprint arXiv:2304.07590* (2023).

[11] Filip Drobnjaković, Pavle Subotić, and Caterina Urban. 2022. Abstract interpretation-based data leakage static analysis. *arXiv preprint arXiv:2211.16073* (2022).

[12] Yunhe Feng, Sreecharan Vanam, Manasa Cherukupally, Weijian Zheng, Meikang Qiu, and Haihua Chen. 2023. Investigating Code Generation Performance of Chat-GPT with Crowdsourcing Social Data. In *Proceedings of the 47th IEEE Computer Software and Applications Conference*. 1–10.

[13] Md Asraful Haque and Shuai Li. 2023. The Potential Use of ChatGPT for Debugging and Bug Fixing. *EAI Endorsed Transactions on AI and Robotics* 2, 1 (2023), e4–e4.

[14] Andrew Head, Fred Hohman, Titus Barik, Steven Drucker, and Robert DeLine. 2019. Managing Messes in Computational Notebooks. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. ACM. doi:10.1145/3290605.3300500

[15] Brahim Hnich, Ali Ben Mrad, Abdoul Majid O Thiombiano, and Mohamed Wiem Mkaouer. 2025. From apologies to insights: extracting topics from chatgpt apologetic responses. *Journal of Decision Systems* 34, 1 (2025), 2438610.

[16] Jingmei Hu, Jiwon Joung, Maia Jacobs, Krzysztof Z. Gajos, and Margo I. Seltzer. 2020. Improving Data Scientist Efficiency with Provenance. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. 1086–1097.

[17] Sayash Kapoor and Arvind Narayanan. 2023. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* (Aug. 2023). doi:10.1016/j.patter.2023.100804 Publisher: Elsevier.

[18] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences* 103 (2023), 102274.

[19] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 6, 4 (2012), 1–21.

[20] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah Alomar. 2025. Software Refactoring Research with Large Language Models: A Systematic Literature Review. *Journal of Systems and Software* (2025), 112762.

[21] Nascimento Nathalia, Alencar Paulo, and Cowan Donald. 2023. Artificial Intelligence vs. Software Engineers: An Empirical Study on Performance and Efficiency using ChatGPT. In *Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering*. 24–33.

[22] Issam Oukay, Moataz Chouchen, Ali Ouni, and Fatemeh Hendijani Fard. 2025. On the Performance of Large Language Models for Code Change Intent Classification. In *IEEE International Conference on Software Analysis, Evolution and*

Reengineering (SANER).
[23] David N Palacio, Alejandro Velasco, Daniel Rodriguez-Cardenas, Kevin Moran, and Denys Poshyvanyk. 2023. Evaluating and Explaining Large Language Models for Code Using Syntactic Structures. *arXiv preprint arXiv:2308.03873* (2023).
[24] Jeffrey Perkel. 2018. Why Jupyter is data scientists' computational notebook of choice. *Nature* 563 (11 2018), 145–146. doi:10.1038/d41586-018-07196-1
[25] Rohith Pudari and Neil A Ernst. 2023. From Copilot to Pilot: Towards AI Supported Software Development. *arXiv preprint arXiv:2303.04142* (2023).
[26] Xing Qian and Eman Abdullah AlOmar. 2026. Can large language models identify and refactor code clones? An empirical study. *Journal of Systems and Software* 234 (2026).
[27] Luigi Quaranta, Fabio Calefato, and Filippo Lanubile. 2021. KGTorrent: A Dataset of Python Jupyter Notebooks from Kaggle. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE. doi:10.1109/msr52588.2021.00072
[28] Nikitha Rao, Bogdan Vasilescu, and Reid Holmes. 2025. From Overload to Insight: Bridging Code Search and Code Review with LLMs. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) *(FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 656–660. doi:10.1145/3696630.3728518
[29] Palash R Roy, Ajmain I Alam, Farouq Al-omari, Banani Roy, Chanchal K Roy, and Kevin A Schneider. 2023. Unveiling the potential of large language models in generating semantic and cross-language clones. *arXiv preprint arXiv:2309.06424* (2023).
[30] Adam Rule, Aurélien Tabard, and James Hollan. 2018. Exploration and Explanation in Computational Notebooks. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (2018). https://api.semanticscholar.org/CorpusID:5048947
[31] Shreya Shankar, Stephen Macke, Sarah Chasins, Andrew Head, and Aditya Parameswaran. 2022. Bolt-on, Compact, and Rapid Program Slicing for Notebooks. *Proceedings of the VLDB Endowment* 15, 13 (2022), 4038–4047. doi:10.14778/3565838.3565855
[32] Yannis Smaragdakis, George Kastrinis, and George Balatsouras. 2014. Introspective analysis: context-sensitivity, across the board. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) *(PLDI '14)*. Association for Computing Machinery, New York, NY, USA, 485–495. doi:10.1145/2594291.2594320
[33] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of chatgpt. *arXiv preprint arXiv:2301.08653* (2023).
[34] Johannes Stümpfle, Devansh Atray, Nasser Jazdi, and Michael Weyrich. 2025. Large Language Model assisted Transformation of Software Variants into a Software Product Line. 12–20. doi:10.1109/ICSR66718.2025.00008
[35] Pavle Subotić, Lazar Milikić, and Milan Stojić. 2022. A static analysis framework for data science notebooks. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 13–22.
[36] Adam Tornhill, Markus Borg, Nadim Hagatulah, and Emma Söderberg. 2025. ACE: Automated Technical Debt Remediation with Validated Large Language Model Refactorings. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) *(FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 1318–1324. doi:10.1145/3696630.3730565
[37] Christoph Treude and Hideaki Hata. 2023. She Elicits Requirements and He Tests: Software Engineering Gender Bias in Large Language Models. *arXiv preprint arXiv:2303.10131* (2023).
[38] John Whaley and Monica S. Lam. 2004. Cloning-based context-sensitive pointer alias analysis using binary decision diagrams. *SIGPLAN Not.* 39, 6 (jun 2004), 131–144. doi:10.1145/996893.996859
[39] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).
[40] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *2014 IEEE Symposium on Security and Privacy (SP)*. IEEE. doi:10.1109/SP.2014.44
[41] Chenyang Yang, Rachel A Brower-Sinning, Grace Lewis, and Christian Kästner. 2022. Data leakage in notebooks: Static detection and better processes. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.

Received 2025-09-12; accepted 2026-03-24